

Database Replication

System Design Deep Dive - Complete Detailed Notes

Every concept explained with an example and a diagram

#	Chapter	Core question it answers
1	What is Database Replication and Why?	Why copy a database?
2	Primary and Replicas in Action	Where do reads and writes go? What if the primary dies?
3	How Replication Works (WAL)	What exactly is copied?
4	Synchronous, Asynchronous, Semi-Synchronous	How long does a write wait?
5	Replication Lag	Why are replicas sometimes behind?
6	Handling Replication Lag	How do we hide or limit staleness?
7	Cheat Sheet and Interview Q&A	What must I remember?

1. What is Database Replication?

Database replication means **copying data from one database to one or more other databases**. The original is the **primary** (leader); the copies are **replicas**.

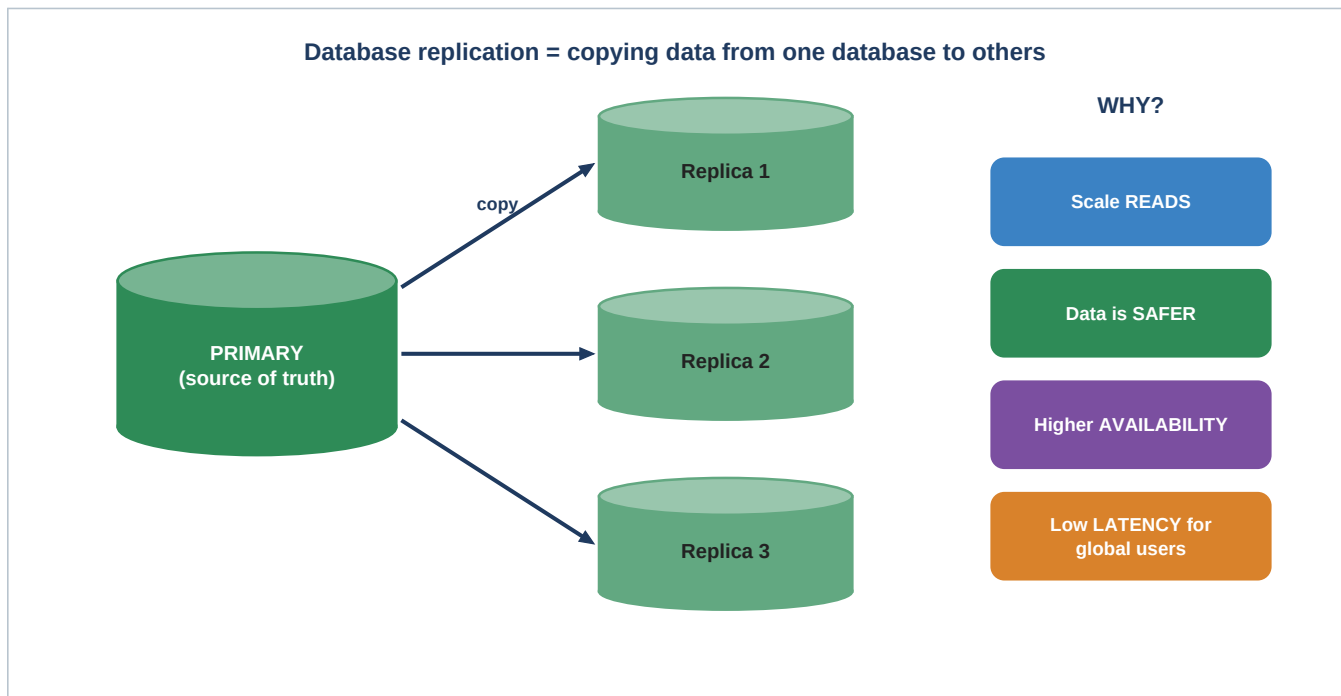


Figure 1: One primary copied to several replicas, and why we do it

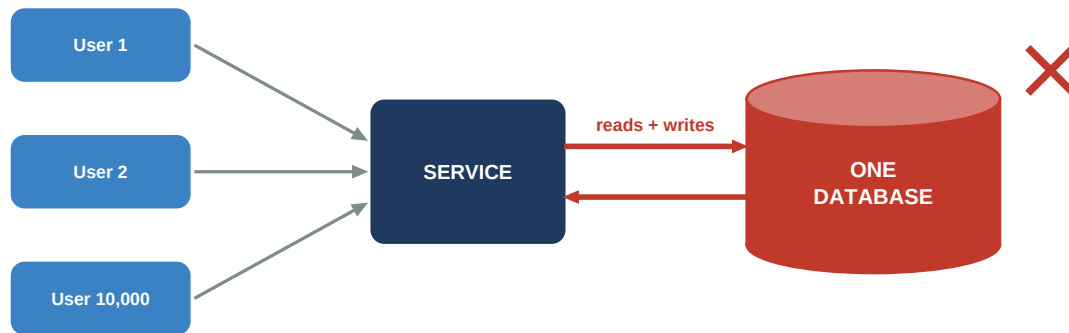
1.1 Why do we need it?

- **Scale reads:** many replicas can answer read queries in parallel. Example: 10,000 users reading a product page are spread across 4 replicas instead of hammering one DB.
- **Data is safer:** if one machine is lost, other copies still exist.
- **Higher availability:** one failure does not kill everything; another copy can take over.
- **Low latency for global users:** a user in Europe reads from a nearby European replica instead of a server across the ocean.

1.2 What happens without replication?

In the traditional architecture a user calls a service and the service talks to the **single database**. With few users it works: user 1 writes X, user 2 reads X. But as users grow to thousands or 10,000, everything funnels into one machine.

Without replication: one database does everything



1) **BOTTLENECK:** thousands of reads AND writes all hit the same machine.

2) **SINGLE POINT OF FAILURE:** if this node crashes, the data is lost / the system is down.

Example: 1,000 users write and 10,000 users read the same product page at once: one DB struggles.

Figure 2: A single database is both a bottleneck and a single point of failure

- **Bottleneck:** all reads and all writes go to the same database, so it becomes overloaded.
- **Single point of failure:** if this node crashes, the entire data is lost (or at least unavailable).

EXAMPLE

On a festival sale day, 1,000 people place orders (writes) while 10,000 browse the same products (reads). One database server gets maxed out; if it dies, the shop is down and recent orders may be gone.

2. Primary and Replicas in Action

2.1 The basic setup

With replication, **writes go to one place (the primary)** but **reads can be served from many places (the replicas)**, and failures become survivable.

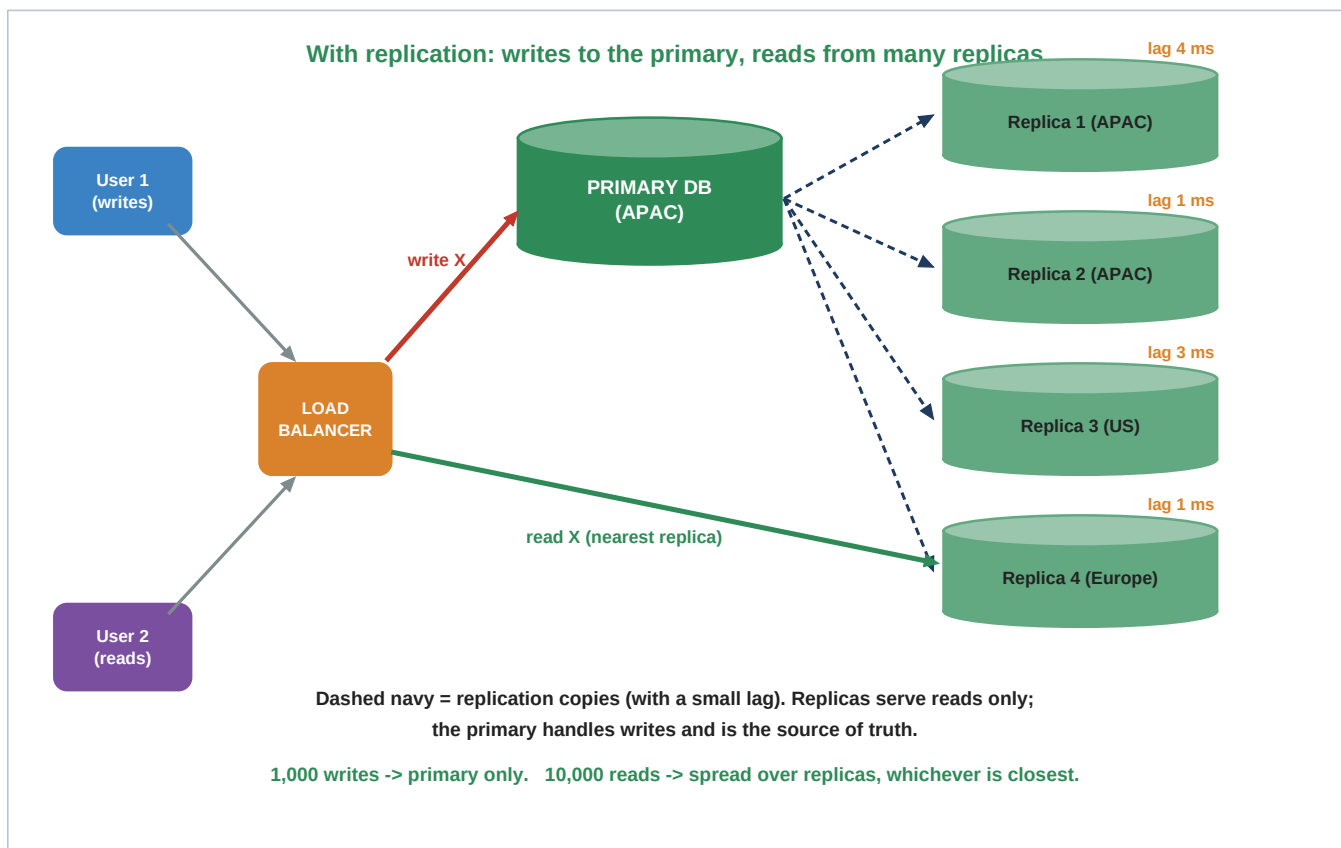


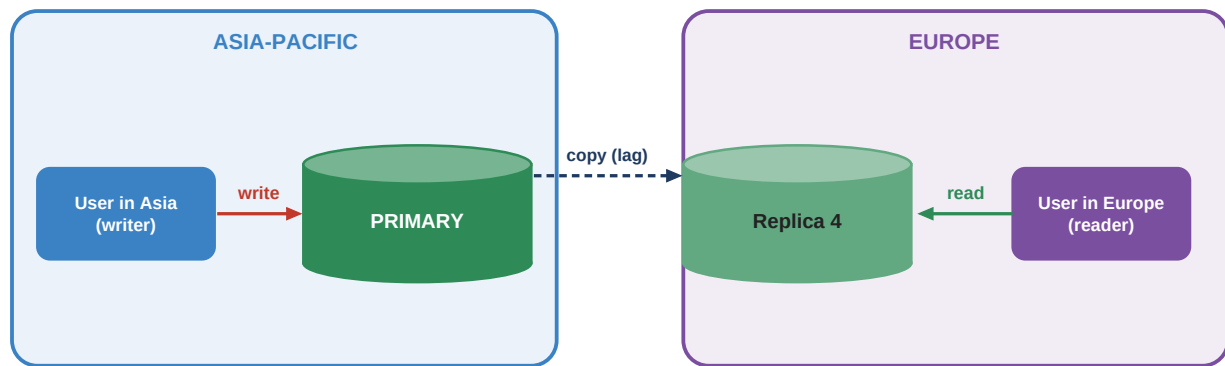
Figure 3: Writes to the primary; reads from the nearest replica; copies arrive with a small lag

- A user sends a **write** (update X) through the load balancer; it goes to the **primary DB**, which updates X.
- The **replicas copy the value** afterwards; each has a small **replication lag** (for example 4 ms, 1 ms, 3 ms, 1 ms).
- A user who wants to **read** X is served by whichever replica is closest.

EXAMPLE

The primary is in APAC. A user in Asia writes X and gets an acknowledgement. A user in Europe reading X is served by the European replica (Replica 4), so the read is quick.

Global users: read from a replica near you



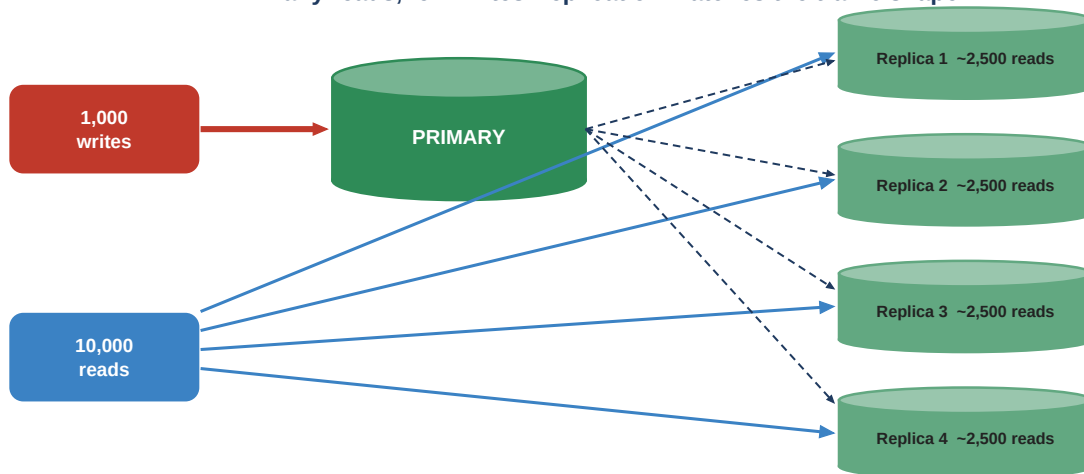
The write is acknowledged by the nearby primary; the European user reads the copy stored in Europe instead of crossing the ocean: low latency (data may be slightly behind).

Figure 4: A European user reads from the European replica

2.2 Replicas are for reads only

- The **primary (leader)** handles **writes** and is the **source of truth**.
- **Replicas** handle **reads** and **copy data from the primary**. That is all they do.

Many reads, few writes: replication matches the traffic shape



Reads are spread over replicas (numbers illustrative); every write still goes to the one primary

Figure 5: 1,000 writes hit the primary; 10,000 reads are spread over replicas

EXAMPLE

1,000 users write and 10,000 users read. All 1,000 writes land on the primary. The 10,000 reads are split across Replica 1-4 (roughly 2,500 each in this illustration), each user served by the closest one. Reads are far more common than writes in most apps, so this fits well.

2.3 What if the primary dies?

Reads outnumber writes, but one server still handles all writes, so it can still be a bottleneck, and if it goes down writes stop. The answer is **leader election**: among the available replicas a **new primary** is chosen.

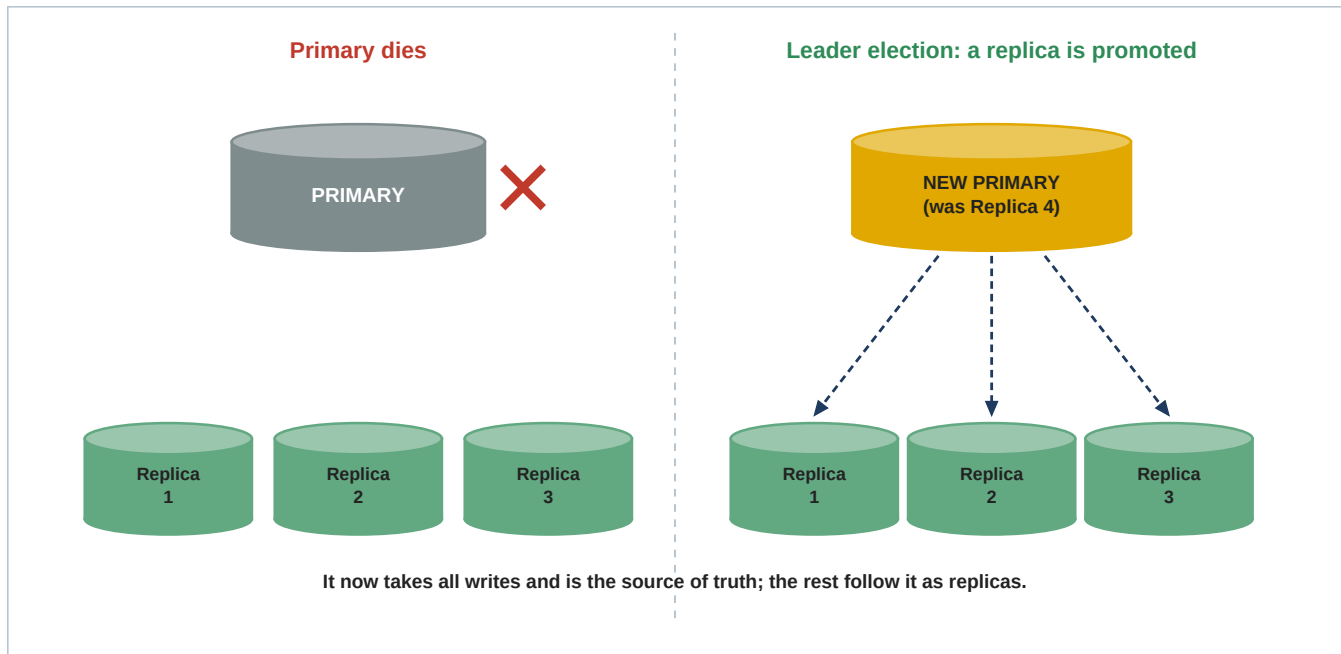


Figure 6: Leader election promotes a replica to primary

EXAMPLE

The primary crashes. Replica 4 is elected. It now handles all writes and is the new source of truth; Replicas 1-3 continue as replicas, now copying from it. (The video notes it covered the leader election algorithm in a separate prerequisite video.)

KEY TAKEAWAY

Basic assumption: exactly one primary at a time, plus multiple read-only replicas.

3. How Replication Works

Step by step: a client writes X to the primary; the primary updates X and responds; then the replicas catch up.

3.1 Write-ahead log (WAL)

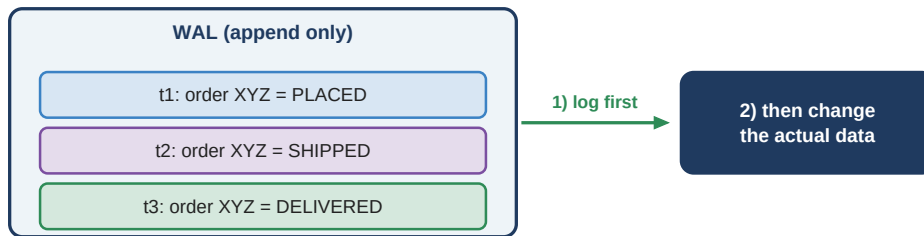
When a request comes in, a database does not simply overwrite the table. It first stores the **action** in a **write-ahead log**: an **append-only log** where the database records **every data change before modifying the actual data**.

A) Normal table: only the LATEST state is kept

order_id	status
XYZ	DELIVERED

placed -> shipped -> delivered: each update overwrites the last
'What was the status 3 days ago?' -> cannot answer

B) WRITE-AHEAD LOG (WAL): an append-only log of every change



Every change is recorded **BEFORE** the data is modified, so history is kept
(‘what was the status 3 days ago?’ = read the log) and replicas can replay the same log.

Key idea: replication does NOT copy the tables themselves; it copies the OPERATIONS from the log, and each replica replays them locally to reach the same final state.

Figure 7: A normal table keeps only the latest state; the WAL keeps every change

EXAMPLE

An order goes placed -> shipped -> delivered. A plain table row ends with just 'delivered'; asking 'what was the status 3 days ago?' is impossible. The WAL has three entries (placed, shipped, delivered), so the history exists, and replicas can replay it.

3.2 Replicas replay the log

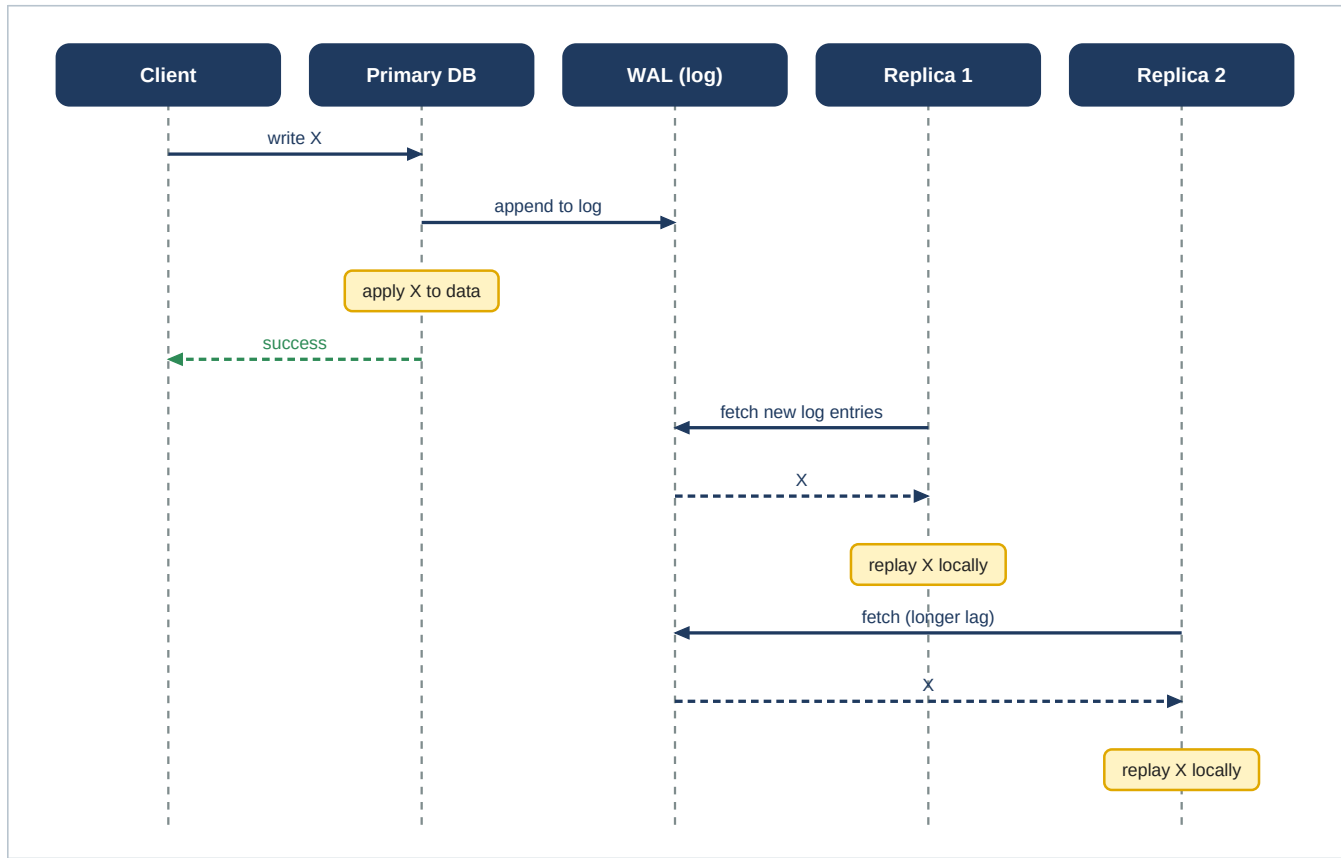


Figure 8: Primary logs and applies the write; replicas fetch and replay it after their own lag

- The user writes X; the primary stores it in the **WAL** and applies it, then returns success.
- **After some replication lag**, each replica gets the new log entries from the primary and **replays the same operations locally** to reach the same final state.
- Replicas **slowly catch up**; how slowly depends on the lag. The result is **eventual consistency**.

KEY TAKEAWAY

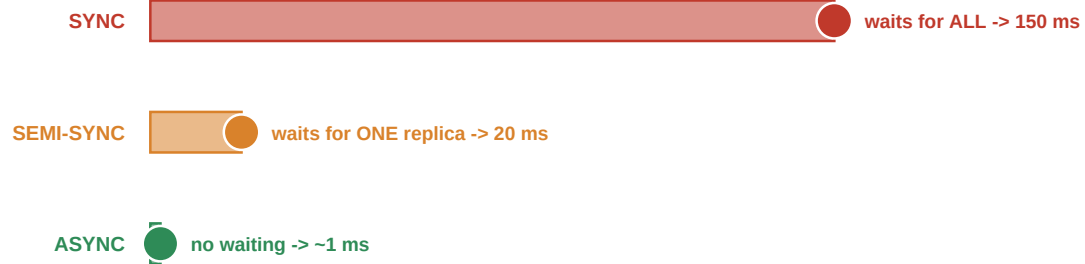
Replication does **not** copy the tables themselves; it copies the **operations**, and each replica replays them.

4. Synchronous, Asynchronous and Semi-Synchronous Replication

The three types differ in **how long the primary waits for replicas before telling the client 'success'**.

When does the client hear 'success'? (illustrative timings)

Replica acks arrive at:



Safer = slower: the more acknowledgements you wait for, the higher the write latency
(with a majority quorum, the sync case would wait for the 2nd-fastest ack, ~60 ms)

Figure 9: The more acknowledgements you wait for, the slower the write

4.1 Synchronous replication (strong consistency)

The primary **waits for replicas to confirm** before it confirms the write to the client. In theory it waits for **all** replicas (with 1,000 replicas, all 1,000 acks).

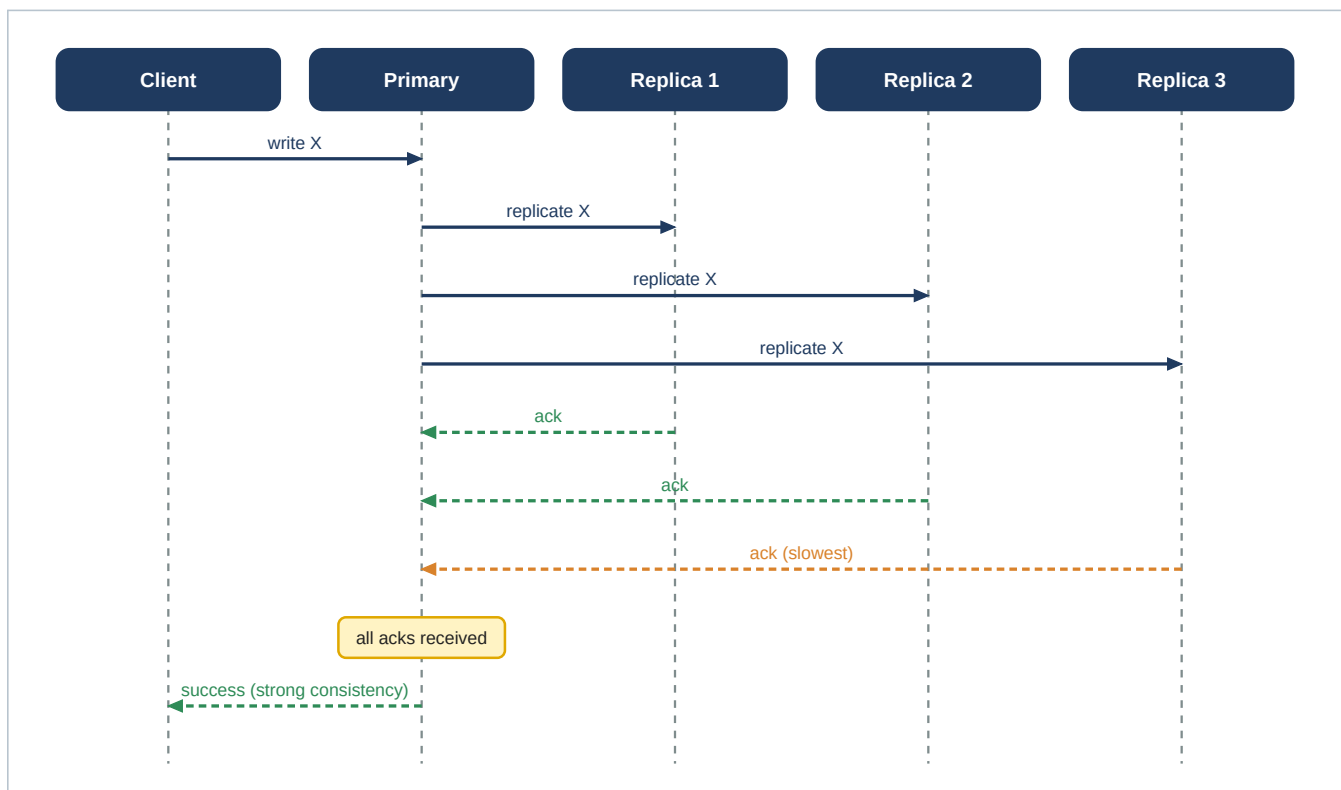


Figure 10: Synchronous: the client hears 'success' only after the replicas have acked

EXAMPLE

Banking and payments: transferring Rs 10,000 means the debited and the credited account must both be safe and in their final state; losing data could be a financial disaster. Used in bank systems, financial transaction ledgers and some configurations of databases such as PostgreSQL and Oracle.

In practice: wait for a majority

Real systems usually wait for $\text{floor}(N / 2) + 1$ nodes rather than every replica. With 3 replicas that is 2. With 5 nodes the leader waits for 3, so the system stays functional even if 2 nodes are down.

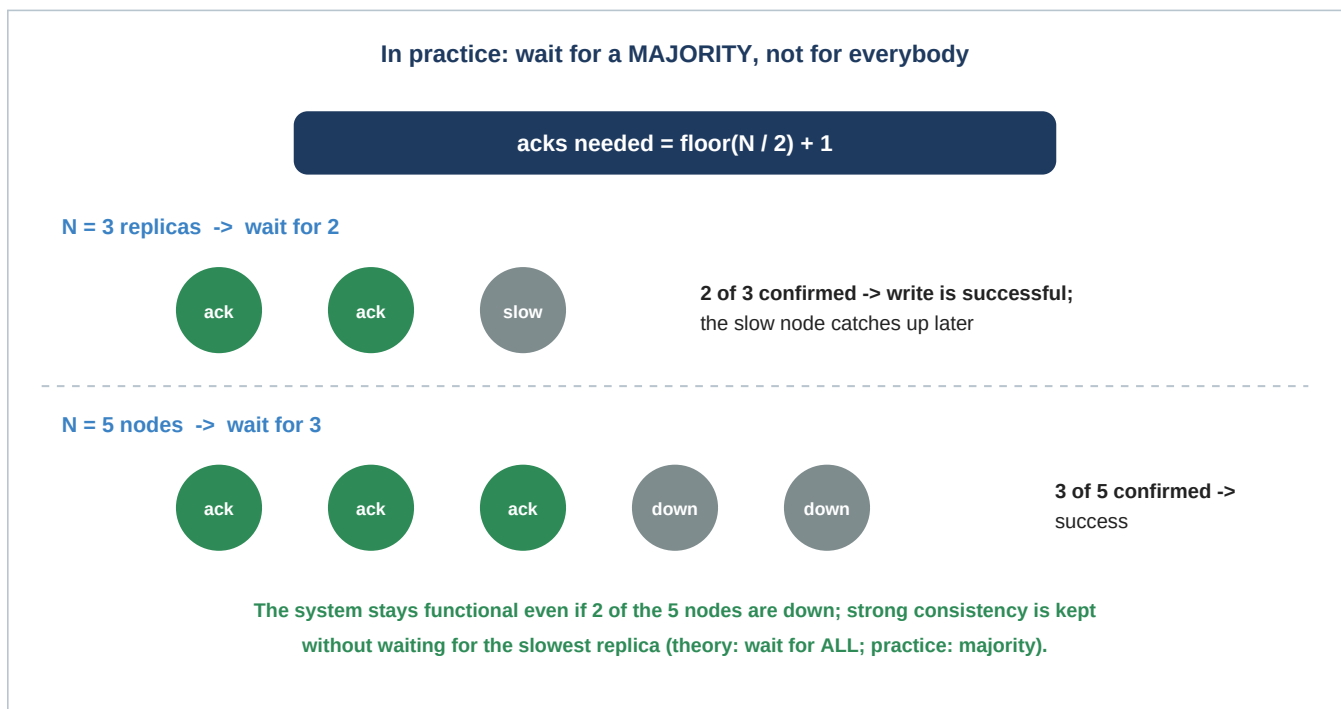


Figure 11: Majority acknowledgements

Trade-offs

- **High write latency:** you wait for the slowest of the required replicas (some replicas have higher lag).
- **Global users suffer:** far-away replicas make every write slow.
- **Never used for social media** style apps.

4.2 Asynchronous replication (very common)

The primary **returns success immediately** after its own write. Replicas catch up later. You get **eventual consistency**, not strong consistency.

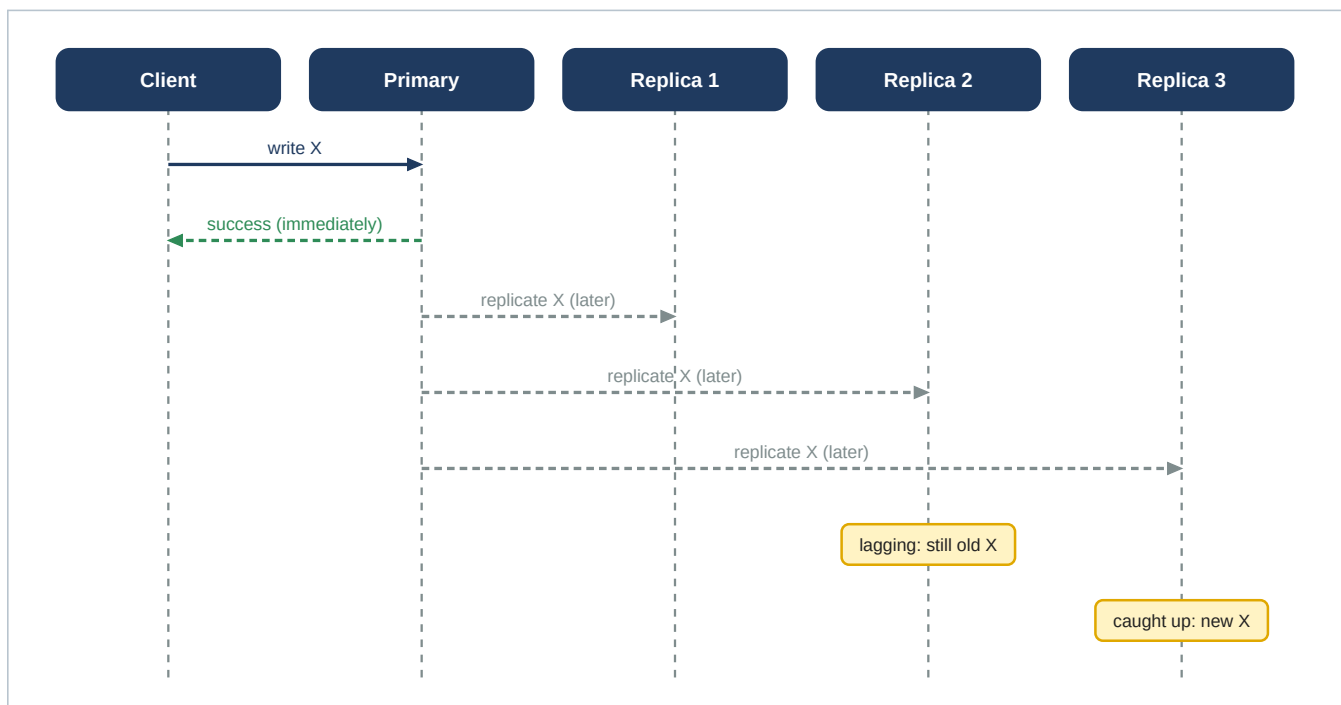


Figure 12: Asynchronous: success first, replication afterwards

EXAMPLE

Instagram posts, tweets, Amazon product views, YouTube comments. You post a photo; the app confirms instantly. A friend on a lagging replica might see it a second later. Nobody is harmed.

- **Why social platforms like it:** it is very fast, scales easily, and losing the last few seconds of data in a crash is acceptable to them.

4.3 Semi-synchronous replication

The primary waits for **at least one replica**, not all. The replica with the **lowest lag** acknowledges first and the primary then answers the client. Latency is higher than asynchronous but much lower than fully synchronous. Other replicas are still eventually consistent.

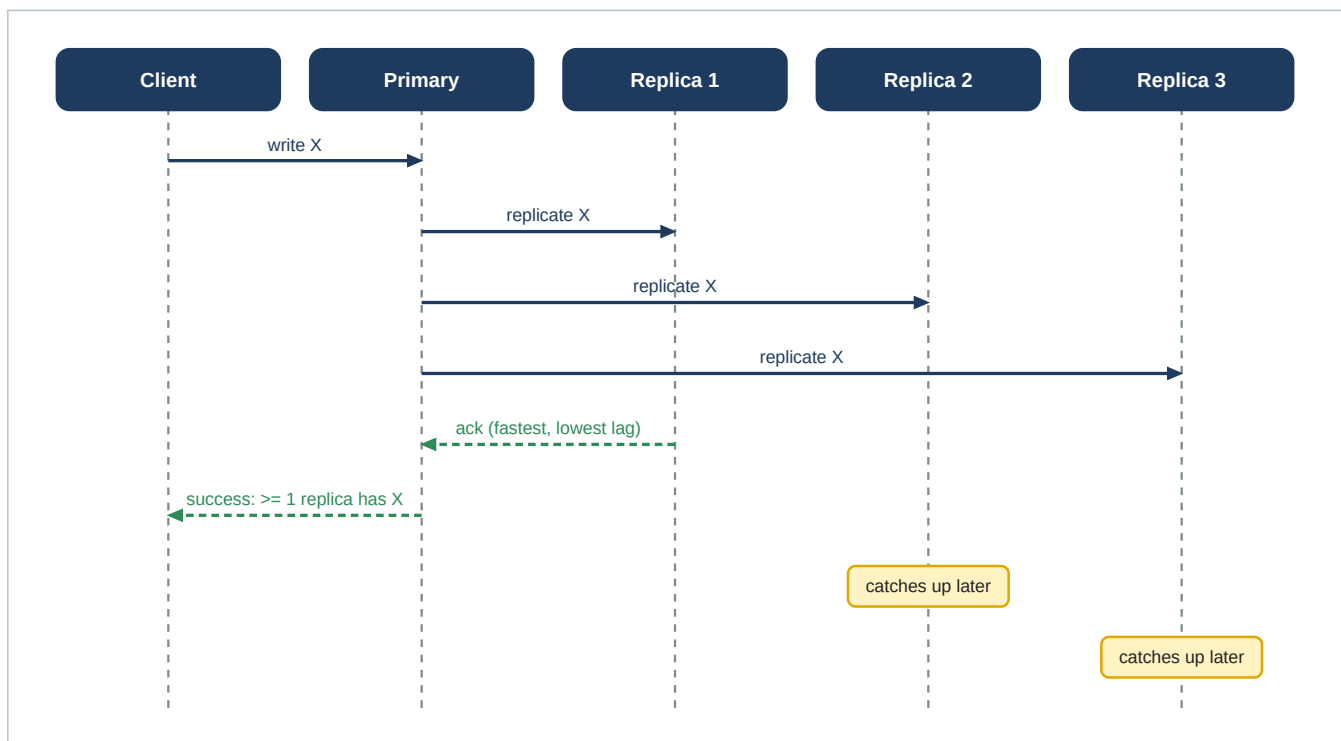


Figure 13: Semi-synchronous: one fast ack is enough

EXAMPLE

E-commerce orders: with 5 replicas Amazon might wait for just one to record the inventory reduction so that two people do not buy the last item, without waiting for all five. It also explains why a product page can look in stock and then say 'out of stock' at checkout: some replicas haven't yet reflected the true state.

- Guarantees **at least one replica has the latest data** with minimal commit delay.
- Suitable for **geographically distributed databases** where full sync would add too much latency.
- **Benefit:** a balance between performance and safety.

4.4 Comparison

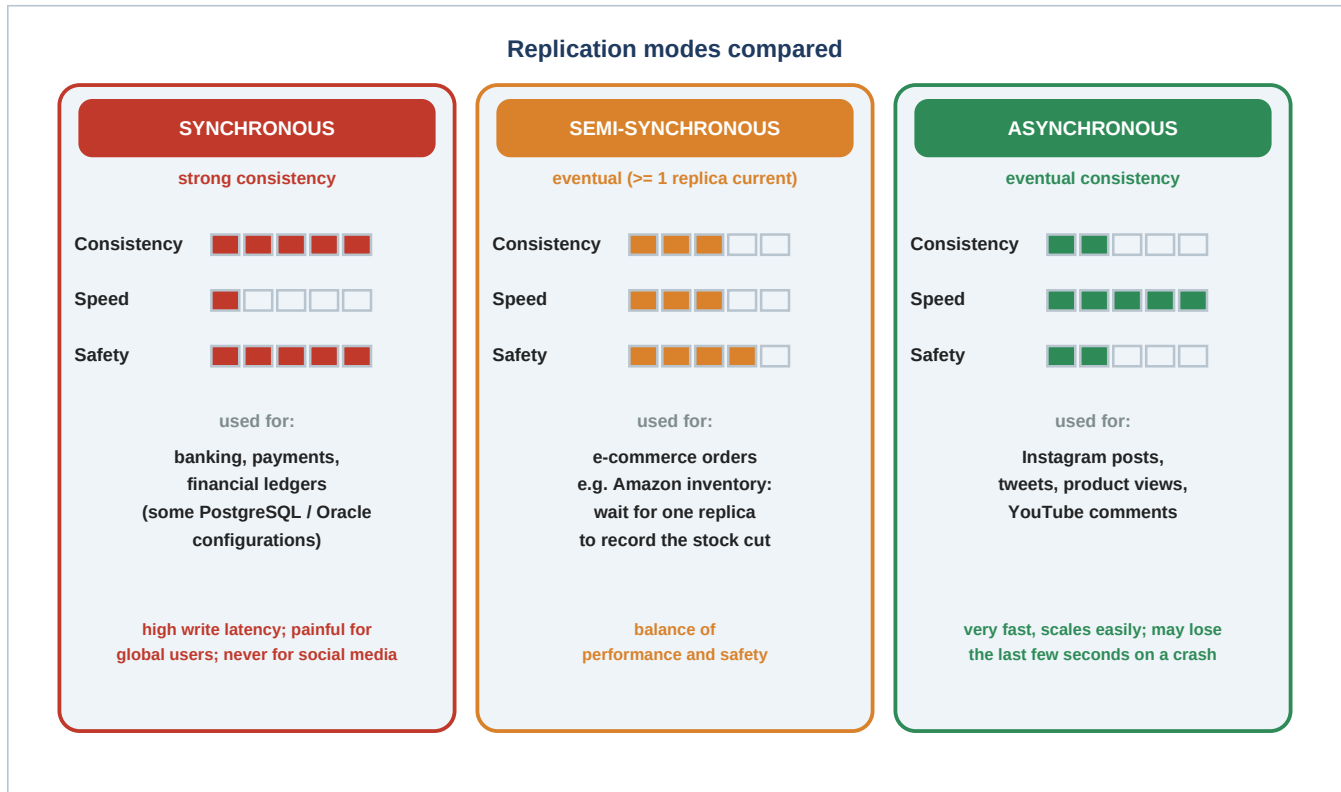


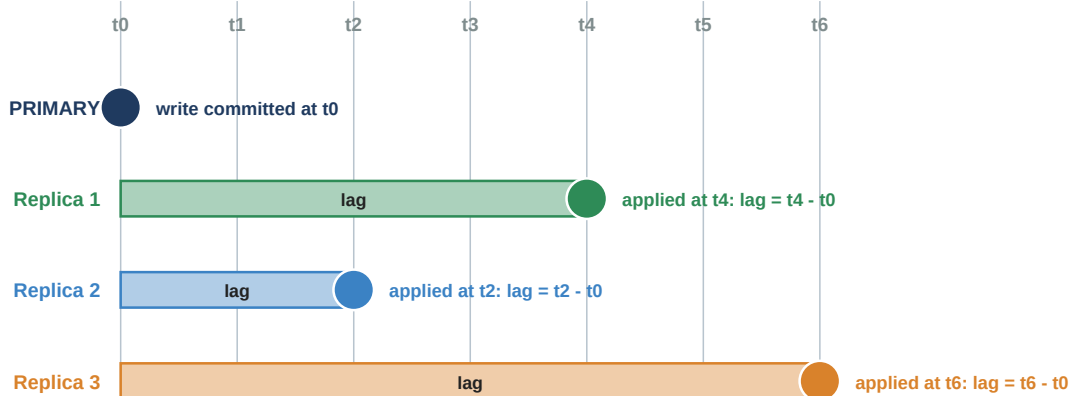
Figure 14: Consistency, speed and safety of each mode

Aspect	Synchronous	Semi-synchronous	Asynchronous
Primary waits for	All (or a majority)	At least one replica	Nobody
Consistency	Strong	Eventual (at least one copy current)	Eventual
Write latency	Highest	Medium	Lowest
Data-loss risk on primary crash	Lowest	Low	Last few seconds possible
Typical use	Banking, payments, ledgers	E-commerce orders, inventory	Social media posts, comments, views

5. Replication Lag

Replication lag is the **time delay** between a write being committed on the primary and the moment it is applied or visible on a replica.

Replication lag = replica apply time - primary commit time



T_0 (the primary commit) is the same for everyone, but every replica applies the change at a different time, so the lag is different per replica. Lag is normal and inevitable (it can't be made zero without killing performance).

Figure 15: Lag differs per replica: $\text{lag} = \text{apply time} - \text{primary commit time}$

- Formula: **replication lag = replica apply time - primary commit time**.
- Example: a write commits on the primary at t_0 and appears on Replica 1 at t_4 : $\text{lag} = t_4 - t_0$.
- Each replica has a **different lag**: t_0 is the same for all, but the apply time differs.

KEY TAKEAWAY

Replication lag is **not a bug**. It is expected, normal and inevitable in a distributed system, especially with asynchronous replication, databases in different regions and heavy write traffic. You cannot make it zero: that would kill performance.

5.1 What causes replication lag?

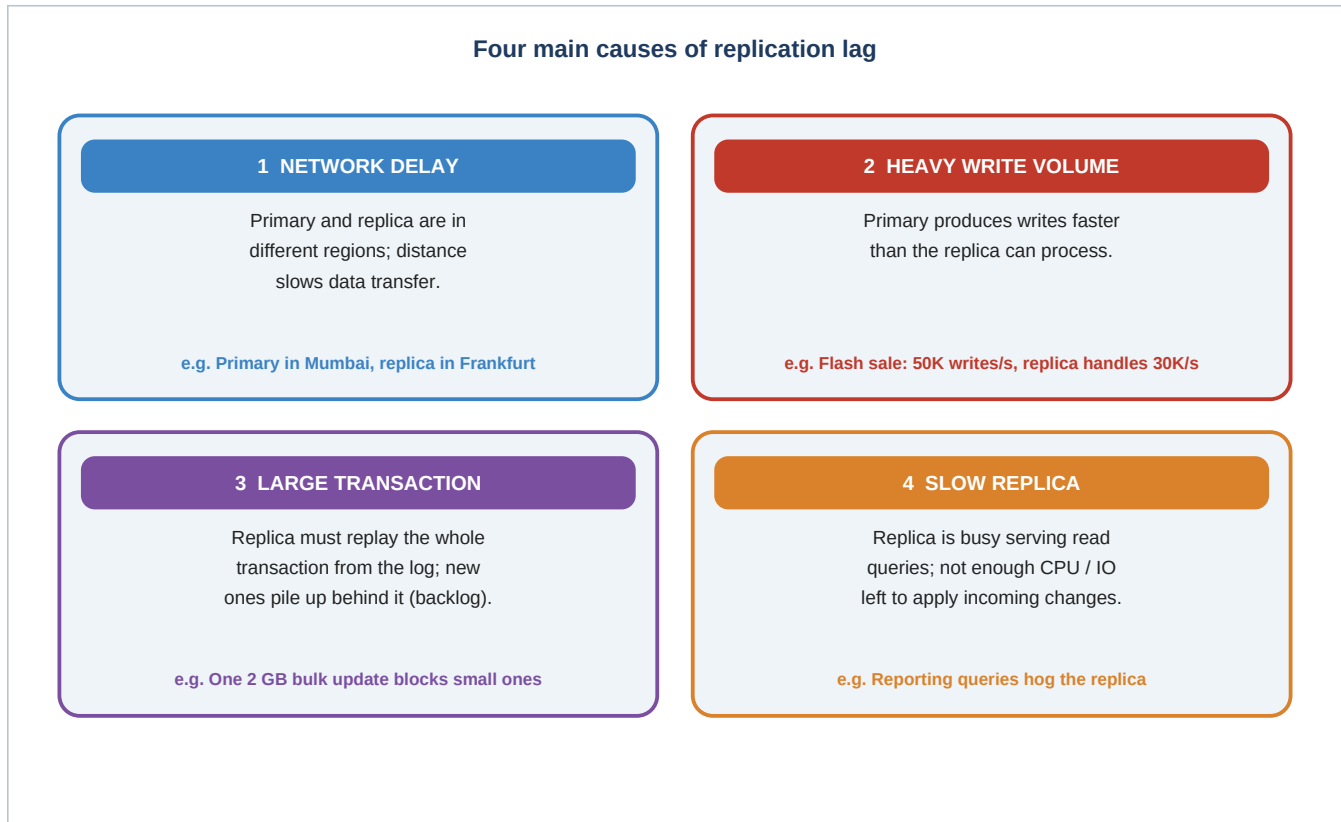


Figure 16: The four main causes

- **Network delay:** primary and replica in different geographic locations; distance slows data transfer. Example: primary in Mumbai, replica in Frankfurt.
- **Heavy write volume:** the primary produces writes faster than the replica can process. Example: a flash sale producing more writes per second than the replica can apply.
- **Large transactions:** the replica must replay all of a big transaction from the log before updating its final state; meanwhile new transactions pile up as a backlog. Example: one huge bulk update.
- **Slow replicas:** a replica busy serving many read queries lacks the CPU and I/O to apply incoming changes from the primary.

6. Handling Replication Lag

Real systems use several techniques; the video covers three popular ones.

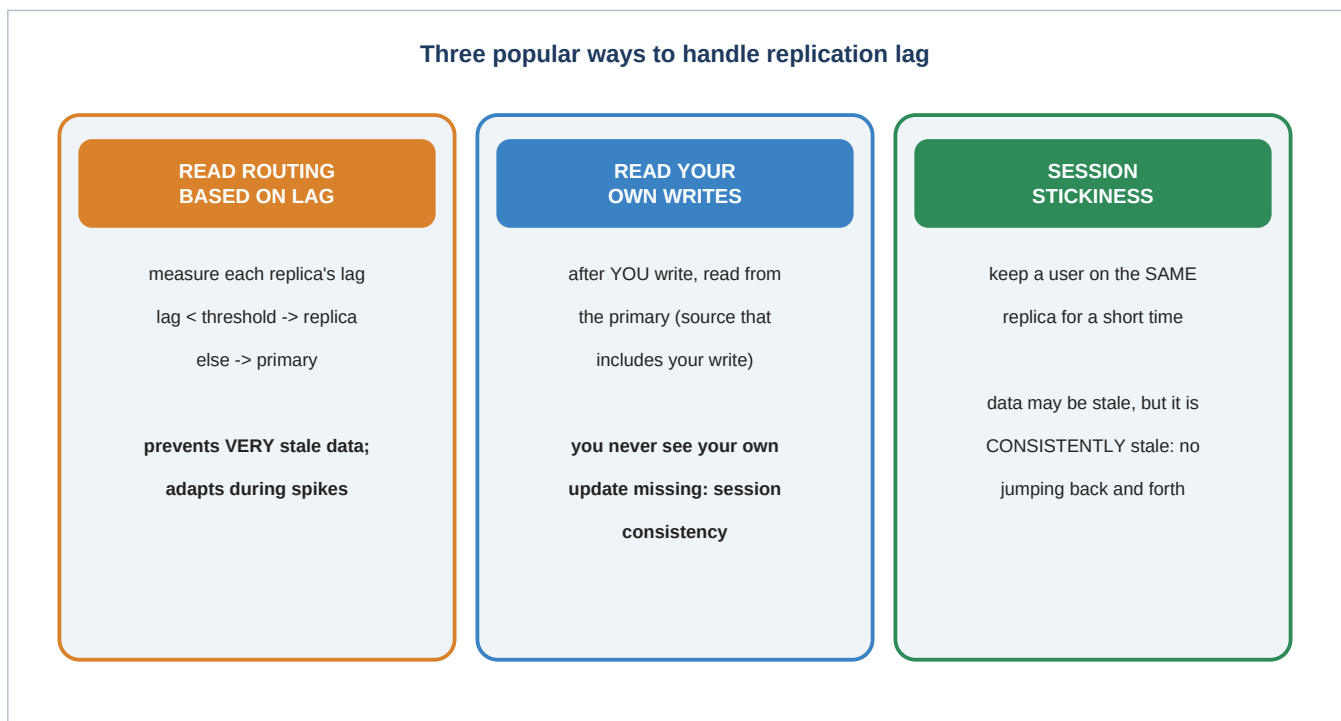


Figure 17: The three techniques side by side

6.1 Read routing based on lag

Do not read from replicas that are too far behind. Continuously **measure** each replica's lag, define a **threshold** and **route reads dynamically**: if the replica's lag is less than the threshold, serve the read from the replica; otherwise route it to the primary.

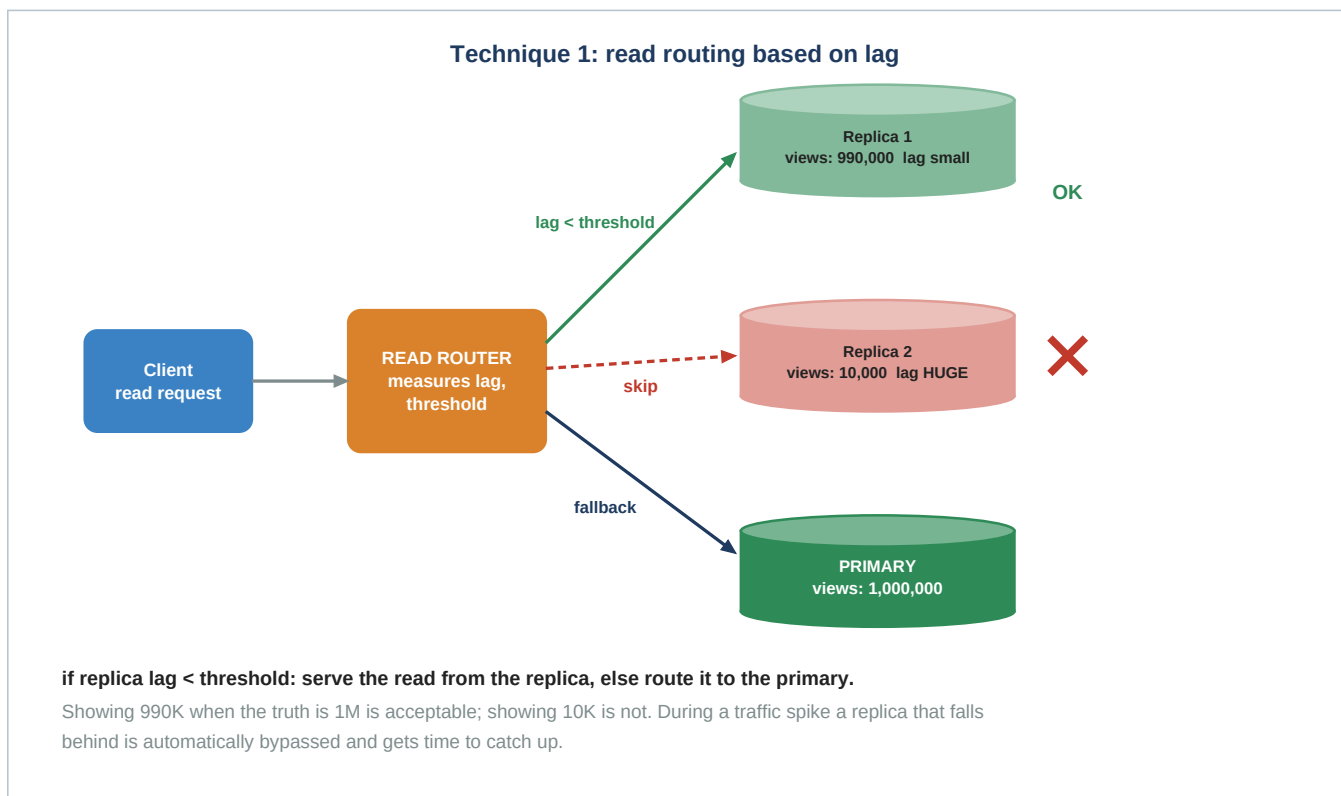


Figure 18: Reads skip a replica that has fallen too far behind

- **Prevents serving very stale data.** Example: my video really has 1,000,000 views. A replica showing 990,000 is acceptable; one showing 10,000 is not, so reads avoid it.
- **Adapts automatically during spikes:** if a traffic spike leaves a replica far behind, users are pointed to the primary instead, and the replica gets time to catch up.

CLARIFICATION

The video stresses that here 'lag' means *how far behind the replica's data is compared with the primary*, not just a time number. In practice systems measure it as time or as a distance in the replication log (for example how many log entries/bytes are still to be applied) and compare it to the threshold.

6.2 Read-your-own-writes

After a user **writes**, that user's reads should come from the **primary** (a source that includes the write). Other users can still read from replicas that may not have caught up. The writer never sees their own update missing: this gives **session consistency**.

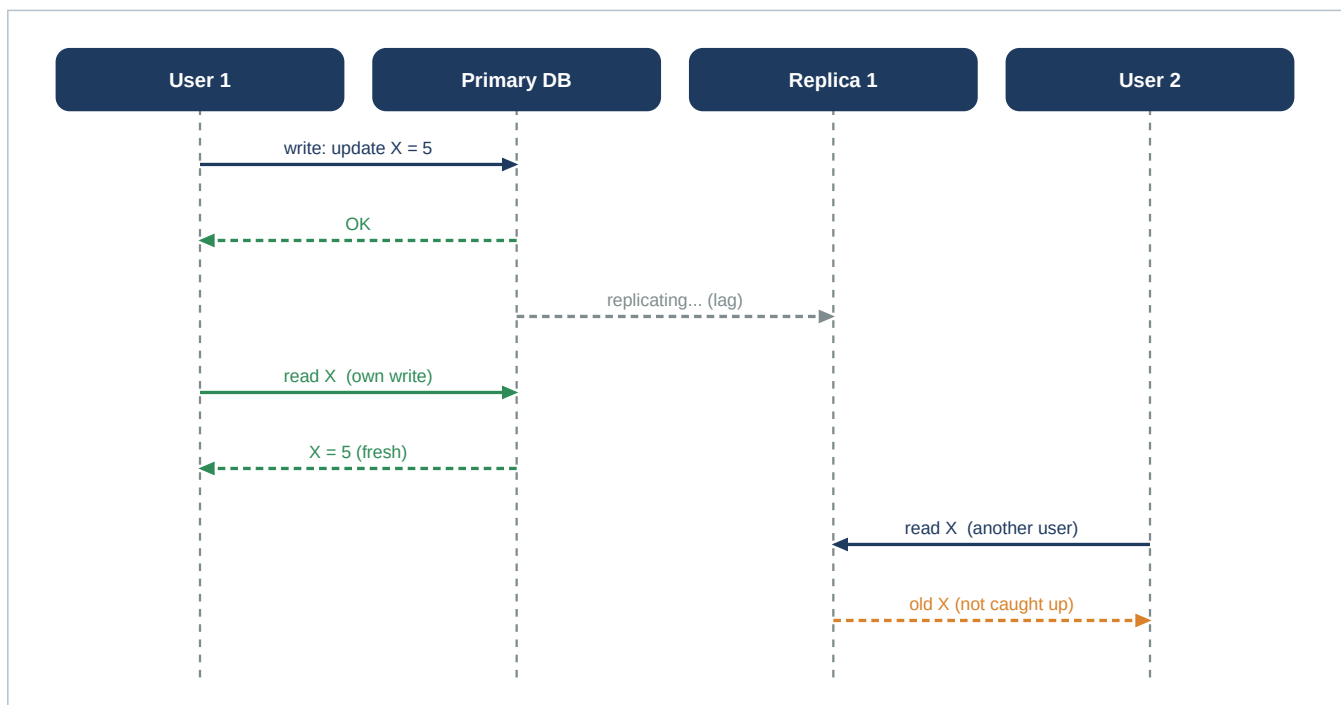


Figure 19: The writer reads from the primary; other users may read from a lagging replica

EXAMPLE

You edit your profile name and press Save. If the next page load came from a lagging replica you would see your old name and think the save failed. With read-your-own-writes your next read goes to the primary and shows the new name; your friends keep reading from replicas and may see the old name for a moment.

6.3 Session stickiness

Keep a user tied to the same database node for a short time, which avoids the confusing effect of different replicas having different lag.

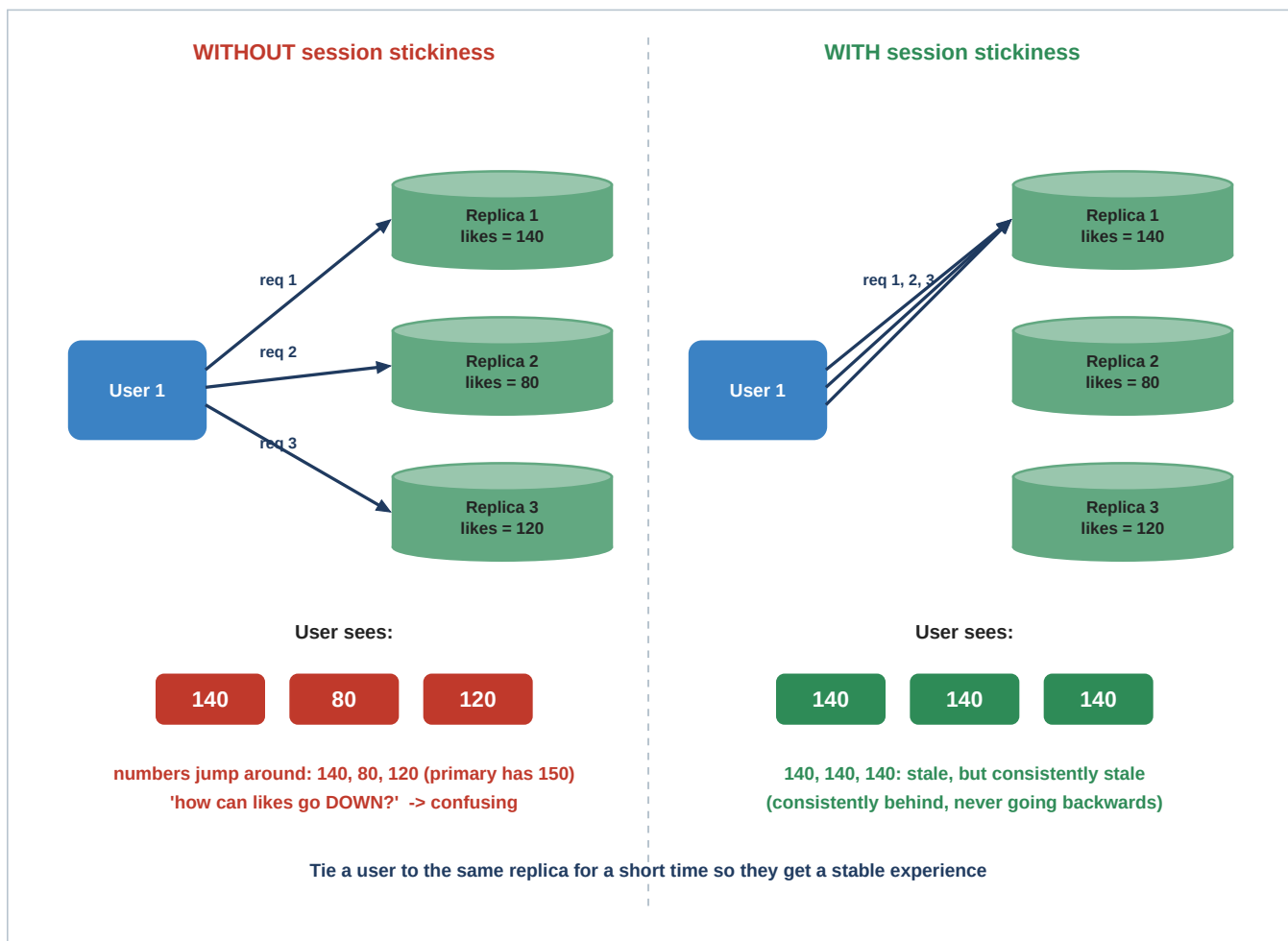


Figure 20: Sticking to one replica gives consistently stale (not jumping) data

EXAMPLE

The primary has 150 likes. Replica 1 shows 140, Replica 2 shows 80, Replica 3 shows 120. Without stickiness your three refreshes hit three replicas and you see 140, 80, 120: likes going down and up! With stickiness all three go to Replica 1 and you see 140, 140, 140. The data is stale, but **consistently stale**: you are consistently behind rather than seeing time go backwards.

Technique	Idea	Guarantee	Example
Lag-based read routing	Skip replicas beyond a lag threshold; use primary	Avoids very stale reads	Avoid the replica showing 10K views of a 1M-view video
Read your own writes	After writing, read from the primary	Session consistency for the writer	See your updated profile name immediately
Session stickiness	Same replica for a short time	Consistently stale, not jumping	Likes read 140, 140, 140 instead of 140, 80, 120

7. Cheat Sheet and Interview Q&A

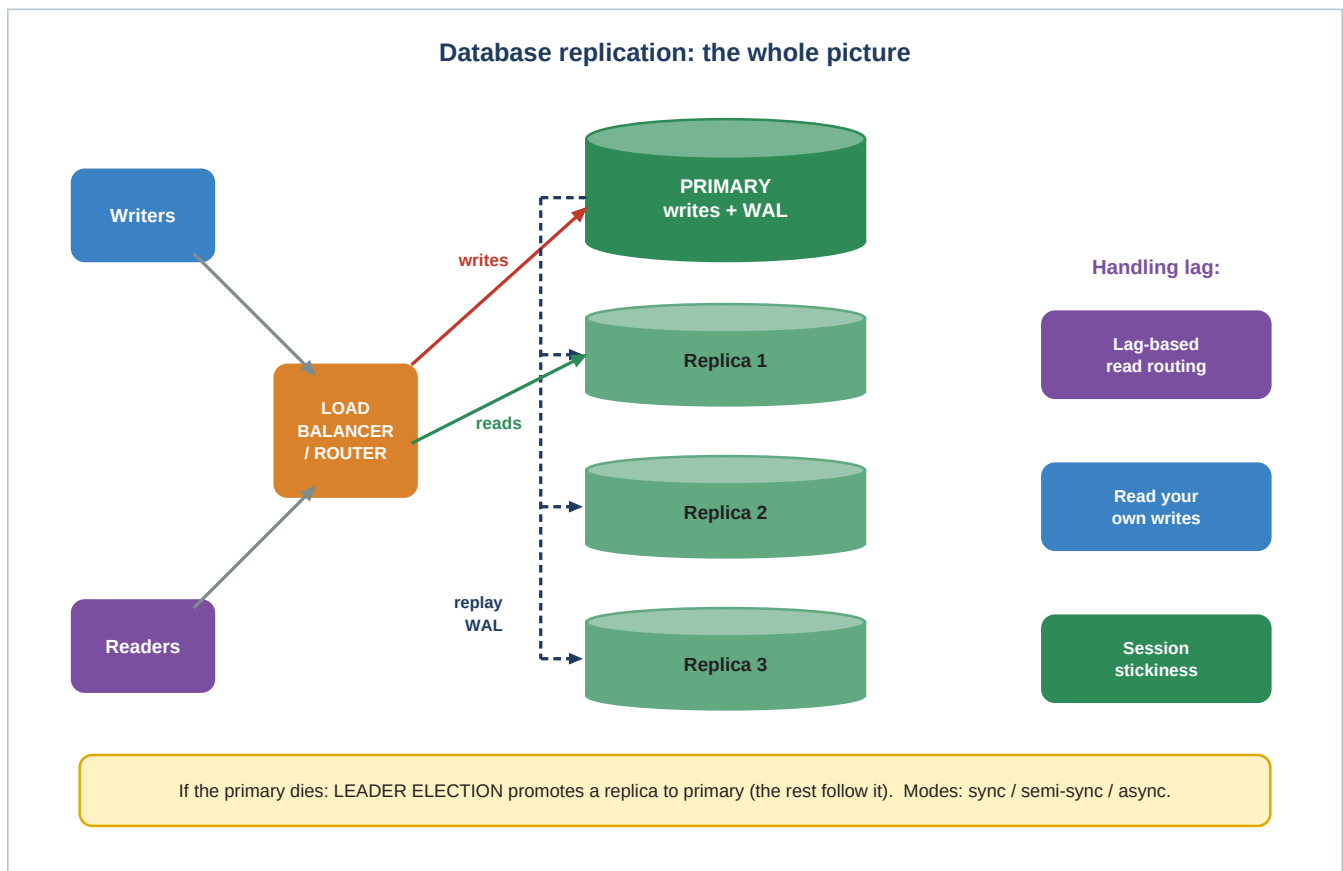


Figure 21: Replication at a glance

7.1 One line per topic

Topic	Remember this
Replication	Copy data from a primary to replicas: scale reads, safer data, availability, lower latency.
Without it	One DB = bottleneck + single point of failure.
Roles	Primary: writes, source of truth. Replicas: reads only, copy from primary.
Primary dies	Leader election promotes a replica.
WAL	Append-only log of every change before data is modified; replicas replay it (operations, not tables).
Synchronous	Wait for replicas (majority in practice: $\text{floor}(N/2)+1$); strong consistency; slow; banking.
Asynchronous	Return immediately; eventual consistency; fast; social media.
Semi-synchronous	Wait for at least one replica; balance; e-commerce orders.
Replication lag	Replica apply time - primary commit time; normal and inevitable; differs per replica.
Lag causes	Network delay, heavy write volume, large transactions, slow replicas.
Lag-based routing	Replica lag < threshold: read from replica; else from primary.
Read your own writes	Read from the primary after you write: session consistency.
Session stickiness	Same replica for a short time: consistently stale instead of jumping.

7.2 Interview Q&A

Q: What is database replication and why use it?

A: Copying data from a primary to replicas to scale reads, keep data safe, improve availability and give global users low latency.

Q: What problems does a single database have?

A: A bottleneck for reads and writes, and a single point of failure.

Q: Where do writes and reads go?

A: Writes to the primary (source of truth); reads to replicas, usually the closest one.

Q: Can you write to a replica?

A: No. Replicas only serve reads and copy from the primary.

Q: What if the primary fails?

A: Leader election chooses a replica as the new primary.

Q: What is a write-ahead log?

A: An append-only log in which the DB records every change before modifying the data; replicas replay it.

Q: Does replication copy tables?

A: No, it copies operations from the log and replicas replay them.

Q: Explain synchronous replication.

A: The primary waits for replicas' acknowledgements before confirming; strong consistency, high latency; used for banking.

Q: How many acks in practice?

A: A majority: $\text{floor}(N/2)+1$, e.g. 2 of 3 or 3 of 5, tolerating failures of the rest.

Q: Explain asynchronous replication.

A: The primary acknowledges immediately; replicas catch up later; eventual consistency; very fast; social media.

Q: Explain semi-synchronous replication.

A: The primary waits for at least one replica; medium latency; guarantees one up-to-date copy; e-commerce orders.

Q: What is replication lag?

A: The delay between commit on the primary and apply on a replica: apply time minus commit time.

Q: Is replication lag a bug?

A: No. It is expected and inevitable; zero lag would kill performance.

Q: What causes lag?

A: Network delay, heavy write volume, large transactions and slow replicas.

Q: How do you handle lag?

A: Lag-based read routing, read-your-own-writes and session stickiness.

Q: What is read-your-own-writes?

A: After writing, a user reads from the primary so they always see their own update; gives session consistency.

Q: Why session stickiness?

A: So a user sees consistently stale data from one replica instead of numbers jumping between replicas.

7.3 Notes on the source material

- The transcript is auto-generated, so some terms are garbled: 'val' = WAL (write-ahead log), 'tradeables' = tables, 'replica lag / replication line' = replication lag, 'JSQL / Oracle RDC' most likely refers to PostgreSQL and Oracle configurations (such as Oracle RAC / Data Guard); the notes name PostgreSQL and Oracle generally.
- The video first defines lag as a time difference (apply time minus commit time) and later says that for read routing it is 'how far behind' the replica is. Both views are consistent: lag can be measured as time or as distance in the log; a clarification box in chapter 6 explains this.
- For majority quorum, the video says 'number of nodes divided by 2 plus 1' ($3 \rightarrow 2$, $5 \rightarrow 3$); it is a majority rule, written here as $\text{floor}(N/2)+1$.
- Timings and read counts in the diagrams (e.g. 20/60/150 ms acks, ~2,500 reads per replica) are illustrative, not from the video.
- The video says a second, deeper video on replication may follow; topics such as multi-leader and leaderless replication are not covered here.